

Dia 1 python e IA

= atribuição de variável.

Função type:

retorna a descrição do tipo de dado de uma variável.

Declaração de variável:

Não é preciso declarar variáveis antes de atribuí-las e também

Ex: type(number-variable)

não precisamos informar o tipo

int:

dessa variável, inclusive, podemos

type(19,95)

até reatribuir para que se refira

float

a um tipo de dado diferente.

// operação de divisão que retira a parte fracionada e transforma o resultado em um inteiro a//b

Comentário em uma linha.

Bloco de código:

** eleva um número ao expoente do outro $a ** b \rightarrow a^b$

Não se usa chaves ("{}") em python para separar blocos de código, se usa apenas (":") para indicar um novo bloco e, após isso, tudo que estiver indentado fará parte do bloco.

Ordem das operações:

Segue a mesma ordem da matemática.

Ex: $-3 + 5 * 2 = 5$

Ex: if (variável == 0):

Função min/max:

variável = -1.

retorna o mínimo menor ou o maior de seus argumentos

* Multiplicação como em C.

Ex: print(min(1,2,3))

1

Operator overloading

Função abs:

A depender dos tipos das variáveis os operadores aritméticos "*" e "+" assumem significados diferentes.

retorna o valor absoluto (módulo).

Função int/float:

Ex: word = "OK" * 3

Pode ser usada para mudar o tipo do argumento (funciona para string também)

print(word)

Ex: print(int(3,14))

OK OK OK

3

print(int("804") + 1)

805

Swap in one line

Ex: $a, b = b, a \rightsquigarrow b = a$ e $a = b$

Funções em python:

Para criar uma função usamos `def` diferente de C; não precisamos retornar um tipo específico.

Função Help:

Mostra uma descrição da função usada como parâmetro. Podemos dar descrições às nossas próprias funções usando `"""description"""` no começo da função.

Ex: `def minValor(a, b, c):`
 `"""retorna o menor valor dos`
 `parâmetros"""`
 `help(minValor)`

[...]

Função como argumento:

Em python podemos chamar uma função usando outra como parâmetro.

Ex: `def call(fn, arg)`
 `return fn(arg)`
`call`
`print(mult-by-five, 1)`

5

Função len:

retorna o tamanho da lista em um inteiro

Função sorted:

Coloca os elementos da lista em ordem alfabética e numérica crescente

Função sum:

Soma os elementos da lista

(max e min): Podem ser usados em listas com.

Funções embutidas:

Em python é como se toda variável fosse um objeto, na qual contém métodos e atributos vinculados a elas.

Remover e Adicionar

- `append`: Adiciona um item ao final da lista.

- `pop()`: Remove o último item e o retorna.

-

Buscar

- `index()`: Procura pelo ^{item} valor na lista

OBS: Para saber se um item existe na lista usamos `in`

Ex: `"Earth" in planets`
`true`

Listas: São basicamente vetores.

Usamos colchetes para criar e acessar no índice.

Ex: `planets = ["Earth", "Mars", "Jupiter"]`

Tuples: A syntax para criar é a mesma da lista, porém usamos parênteses. São imutáveis. São usados geralmente para retornar valores que passam de 1 em quantidade.

Ex: `x = 0,125`

`x.as_integer_ratio()`
(1,8) $\rightarrow$ numerador e denominador.

`numerador, denominador = x.as_integer_ratio()`
`print(numerador/denominador)`
0,125

Loops de repetição:

for: Usamos o for da seguinte maneira, ao invés do for ter um contador, como em C, ele funciona percorrendo itens (iterador).

Ex: `for char in strWord:`

`... #percorre cada caractere`

`for i in range(5):`

`... #percorre a sequência em range`

Função Range:

Gera uma sequência de números

Ex: `for i in range(2):`

`print(i)`

0 $\rightarrow$ 1

while: funciona exatamente igual em linguagem C.

List comprehensions:

Criar uma lista baseada em loops e condições.

Ex: `squares = [n**2 for n in range(10)]`
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

`short_planets = [planet for planet in planets if len(planet) < 6]`
["Venus", "Earth", "Mars"]

~~Podemos usar para construir em mais de uma linha sem interferir na sintaxe.~~

Ex: `print('Hello world')`

Passa a ser considerado os espaços

Funções para strings: (métodos)

- `upper()`: Deixa tudo maiúsculo

- `lower()`: Deixa tudo minúsculo

- `startswith()`: Verifica se a string começa com o = ao argumento.

- `endswith()`: Verifica se a string termina = ao argumento

- `split()`: Divide a string, por padrão, nos espaços.

- `join()`: Transforma tudo em uma longa string, utilizando a string sobre a qual foi chamado como separador

Ex: `'/'.join([month, day, year])`
"01/31/1956"

+ Podemos usar o operador para concatenar strings

Ex: planet + ', we miss you'
'Pluto, we miss you.'

Função str(): Transforma o tipo para string.

• Format: Pode concatenar usando chaves para posicionar outras strings

Ex: "23 is a planet".format(planet) ~> 'Pluto is a planet'

Dicionários: Podemos criar chaves com valores, que funcionam parecido com as listas.

Ex: numbers = {'one': 1, 'two': 2}

numbers['one'] ~> 1

numbers['two'] = 'Pluto'

['one': 1, 'two': 'Pluto']

• Keys: Mostra as chaves

• Values: Mostra os valores associados

import: Para importar funções usamos import.

Ex: import math import math as mt
print(math.pi)

3,1415...